



Introdução a Embeddings e Armazenamentos de Vetores

O papel dos embeddings no aprendizado de máquina, como criá-los, como gerenciá-los em escala e aplicações no mundo real

12 minutos de leitura · 3 de dezembro de 2024



Aleix López Pascual

Seguir



Ouvir



Compartilhar



Mais

Com o surgimento dos **Large Language Models (LLMs)**, a capacidade de processar e analisar com eficácia diversos tipos de dados (imagens, vídeo, texto, áudio e outros) tornou-se cada vez mais crucial. Os embeddings surgiram como um pilar fundamental do aprendizado de máquina moderno, oferecendo uma solução poderosa para esse desafio, transformando dados heterogêneos em um formato unificado adequado para uma ampla gama de aplicações. Neste artigo, **exploraremos o fascinante mundo dos embeddings, abordando diversas técnicas de embedding, como gerenciar e consultar embeddings em escala e aplicações do mundo real.**

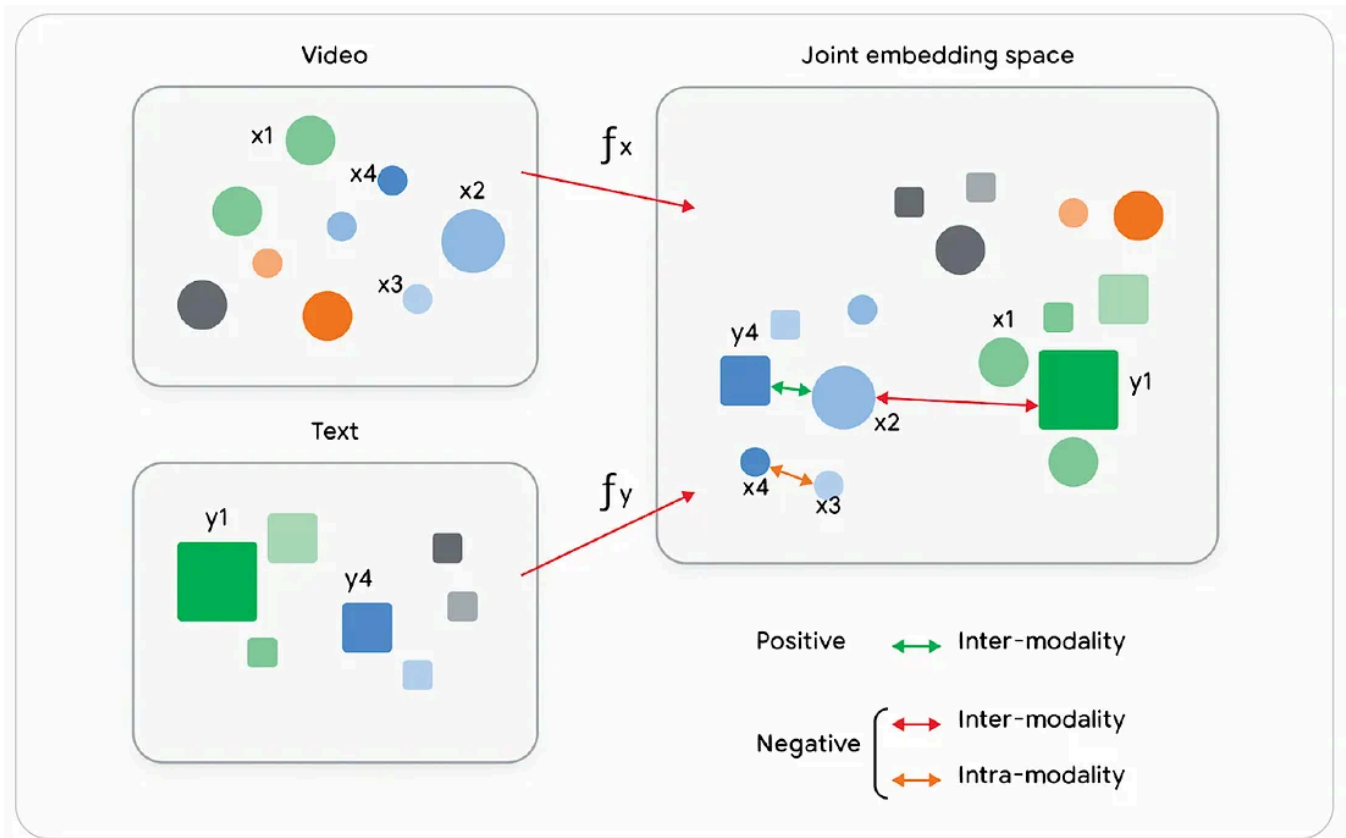
O que são embeddings?

Embeddings são representações vetoriais numéricas de baixa dimensão de dados do mundo real, como texto, imagens, áudio e vídeo. Eles fornecem:

- **Representação compacta** : os embeddings atuam como um meio de compressão com perdas dos dados originais, mantendo suas propriedades essenciais, facilitando o processamento e o armazenamento eficientes de dados em larga escala.
- **Significado semântico** : As distâncias geométricas entre dois vetores no espaço de incorporação refletem as relações entre os objetos do mundo real que

representam. Isso permite comparar a similaridade ou a diferença entre objetos de dados de diferentes tipos em uma escala numérica.

Imagine comprimir uma imagem complexa ou um documento extenso em um vetor numérico conciso que retém seu significado principal — essa é a mágica dos embeddings. Ao projetar diversos tipos de dados em um espaço vetorial comum, os embeddings multimodais podem ser usados diretamente como entrada para modelos de aprendizado de máquina, fornecendo uma representação poderosa dos dados.



Projetar objetos/conteúdo em um espaço vetorial conjunto com significado semântico. Intramodalidade refere-se a relacionamentos ou interações dentro da mesma modalidade. Intermodalidade refere-se a relacionamentos ou interações entre diferentes modalidades.

Tipos de incorporação

Vamos explorar algumas técnicas de incorporação padrão para diferentes tipos de dados:

1. Incorporação de texto

Incorporações de palavras:

Embeddings de Palavras capturam o significado semântico de palavras individuais, aproveitando o princípio de que o significado de uma palavra é definido por suas vizinhas em um corpus. Palavras com significados semelhantes devem ter

embeddings semelhantes. Por exemplo, a palavra "computador" deve ter um embedding semelhante ao de palavras como "laptop" e "PC", mas diferente do de uma palavra como "carro".

Estes são alguns dos algoritmos mais populares:

- **Word2Vec** : Este algoritmo pode ser dividido em duas técnicas diferentes:
 - **A abordagem do saco contínuo de palavras (CBOW)** : Tenta prever a palavra do meio, usando os embeddings das palavras ao redor como entrada. Este método é independente da ordem das palavras ao redor no contexto. Esta abordagem é rápida de treinar e é ligeiramente mais precisa para palavras frequentes.
 - **A abordagem do skip-gram** : A configuração é inversa à do CBOW, com a palavra do meio sendo usada para prever as palavras ao redor dentro de um determinado intervalo. Esta abordagem é mais lenta de treinar, mas funciona bem com dados pequenos e é mais precisa para palavras raras.
- **GloVe (Vetores Globais para Representação de Palavras)** : Ao contrário do Word2Vec, o GloVe não captura apenas estatísticas locais (palavras próximas), mas também as estatísticas globais (palavras em todo o corpus). Ele faz isso criando primeiro uma matriz de coocorrência, que representa as relações entre as palavras. Em seguida, ele usa uma técnica de fatoração para aprender representações de palavras a partir da matriz de coocorrência.
- **SWIVEL (Skip-Window Vectors com Amostragem Negativa)** : Utiliza uma matriz de coocorrência como o GloVe, mas introduz otimizações que tornam o processo de treinamento mais escalável e eficiente. No entanto, é ligeiramente menos preciso que o GloVe.

- **Doc2Vec** : Este modelo, inspirado no Word2Vec, tenta abordar as limitações do BoW tradicional: tanto a ordem das palavras quanto os significados semânticos são ignorados. Para corrigir isso, o Doc2Vec adiciona uma "incorporação de parágrafo" ao modelo Word2Vec. Essa incorporação é treinada juntamente com as incorporações de palavras para prever palavras dentro do documento, permitindo a captura de algumas informações contextuais.

Em contraste, as **abordagens mais recentes** para incorporação de documentos utilizam **modelos de linguagem pré-treinados** , que aproveitam o poder de transformadores e conjuntos de dados em larga escala. Alguns dos algoritmos mais populares incluem:

- **BERT (Representações de Codificador Bidirecional a Partir de Transformadores)** : O BERT foi um modelo inovador que alcançou resultados de ponta em diversas tarefas de PLN em 2018. Trata-se de um modelo de transformador, baseado apenas em codificador, que utiliza um objetivo de modelagem de linguagem mascarada durante o pré-treinamento, o que envolve o mascaramento aleatório de algumas palavras na entrada e o treinamento do modelo para prever as palavras mascaradas com base no contexto circundante. Isso permite que o BERT aprenda representações bidirecionais de palavras.
- **RoBERTa (Abordagem de Pré-Treinamento BERT Robustamente Otimizada)** : RoBERTa é uma variante aprimorada do modelo BERT original. Ele aprimora o desempenho do BERT realizando diversas otimizações durante o pré-treinamento, incluindo mais dados e tempos de treinamento.
- **Sentence-BERT** : O Sentence-BERT foi projetado especificamente para tarefas de incorporação de frases. Ele se baseia na arquitetura do BERT e é ajustado para produzir incorporações de frases semanticamente significativas.
- **SimCSE (Aprendizagem Contrastiva Simples de Embeddings de Frases)** : O SimCSE utiliza uma abordagem de aprendizagem contrastiva para treinar embeddings de frases. Ele gera pares de exemplos positivos e negativos a partir das frases de entrada e treina o modelo para maximizar a similaridade entre pares positivos e minimizar a similaridade entre pares negativos.
- **T5 (Transformador de Transferência de Texto para Texto)** : T5 é um modelo de linguagem abrangente que enquadra todas as tarefas de PNL como problemas de conversão de texto para texto. O T5 utiliza uma arquitetura codificadora-

decodificadora e é conhecido por sua grande versatilidade no tratamento de tarefas de PNL.

Gemini, GPT, Llama ou outras arquiteturas somente de decodificador podem ser usadas para gerar incorporações de documentos?

Em geral, modelos generativos e autorregressivos não são adequados para embeddings, pois sua compreensão da entrada está espalhada por vários estados ocultos. É melhor usar um modelo que tenha sido treinado com o propósito específico de produzir embeddings. Normalmente, trata-se de um codificador transformador e, nesses casos, você obtém o estado oculto da última camada do último token de "fim de sequência". Isso significa que a compreensão do modelo está concentrada em um único lugar. Mais detalhes podem ser encontrados neste artigo:

arXiv:2201.10005v1 [cs.CL] 24 de janeiro de 2022

2. Incorporação de imagens

Um dos métodos mais populares para derivar embeddings de imagens é treinar um modelo CNN ou Vision Transformer em uma tarefa de classificação de imagens em larga escala (por exemplo, ImageNet) e, em seguida, usar a penúltima camada como embedding de imagem. Se bem-sucedido, essa camada terá aprendido alguns mapas de características discriminativas importantes. Esses mapas de características representam padrões visuais aprendidos que são cruciais para a classificação de imagens ou outras tarefas de imagem.

3. Embeddings multimodais

Para obter embeddings multimodais, embeddings unimodais individuais (por exemplo, embeddings de texto e embeddings de imagem) são combinados, levando em consideração as relações semânticas entre as modalidades. Essas relações são normalmente aprendidas por meio de um processo de treinamento separado que alinha os embeddings em um espaço latente compartilhado.

Esse processo garante que dados de diferentes modalidades (texto, imagens, áudio, vídeo etc.) sejam representados em um vetor semântico de tamanho fixo no mesmo espaço latente, permitindo comparação e integração diretas. O treinamento frequentemente envolve técnicas como **aprendizagem contrastiva**, em que pontos de dados pareados (por exemplo, uma imagem e sua legenda associada) são aproximados no espaço de incorporação, enquanto pares não correspondentes são afastados. Esse alinhamento captura a semântica intramodal (por exemplo,

relacionamentos dentro de texto ou dentro de imagens) e intermodal (por exemplo, relacionamentos entre texto e imagens).

4. Incorporação de dados estruturados

Ao lidar com dados estruturados — ou seja, dados organizados em linhas e colunas — é necessário criar um modelo de incorporação personalizado, adaptado à sua aplicação específica. Em outras palavras, nesses casos, é raro encontrar um modelo de incorporação pré-treinado que possa ser usado diretamente. Podemos encontrar dois casos comuns:

- **Dados Estruturados Gerais :** Embeddings para dados estruturados, como dados de sensores ou dados tabulares, podem ser gerados usando técnicas de redução de dimensionalidade, como PCA . Esses embeddings podem ser usados para detecção de anomalias e como entradas para tarefas de aprendizado de máquina subsequentes.
- **Dados Estruturados de Usuário/Item :** Este tipo de dado estruturado vai além de uma tabela de dados geral. Inclui múltiplos componentes interconectados: dados do usuário (por exemplo, dados demográficos, preferências ou comportamentos), dados de item/produto (por exemplo, atributos como categoria, preço ou marca) e dados que descrevem a interação entre usuários e itens (por exemplo, avaliações, cliques ou histórico de compras).
O objetivo principal de trabalhar com esses dados é mapear usuários e itens em um espaço de incorporação compartilhado, permitindo que sistemas de recomendação prevejam interações de forma eficaz. Ao representar usuários e itens como embeddings de tamanho fixo, o sistema pode mensurar sua similaridade ou relevância, possibilitando recomendar os itens mais adequados para cada usuário. Uma técnica popular para isso é a **Filtragem Colaborativa** .

5. Incorporações de gráficos:

Embeddings de grafos são uma maneira de representar **nós, arestas ou grafos inteiros como vetores numéricos de tamanho fixo, preservando as informações estruturais e relacionais inerentes ao grafo** . Esses embeddings capturam as propriedades locais (vizinhança) e globais (estrutura geral) do grafo, permitindo diversas tarefas posteriores, como classificação de nós, agrupamento, predição de links, sistemas de recomendação e muito mais.

Tomemos como exemplo uma rede social onde cada pessoa é um nó e as conexões entre as pessoas são definidas como arestas. Usando a incorporação de grafos, você

pode modelar cada nó como uma incorporação, de modo que a incorporação capture não apenas as informações semânticas sobre a pessoa em si, mas também suas relações e associações, enriquecendo assim a incorporação. Por exemplo, se dois nós estiverem conectados por uma aresta, os vetores para esses nós serão semelhantes. Você poderá então prever com quem a pessoa é mais semelhante e recomendar novas conexões.

Algoritmos populares para incorporação de gráficos incluem **DeepWalk**, **Node2vec**, **LINE** e **GraphSAGE** .

Treinamento de incorporação

Modelos modernos de incorporação frequentemente empregam uma **arquitetura de codificador duplo** , também conhecida como **arquitetura de duas torres** , para treinamento. Nessa configuração, duas redes neurais separadas, ou "torres", são usadas para codificar diferentes aspectos dos dados. Por exemplo, para o modelo de incorporação de texto usado em perguntas e respostas, uma torre é usada para codificar as consultas e a outra torre é usada para codificar os documentos. Para o modelo de incorporação de imagem e texto, uma torre é usada para codificar as imagens e a outra torre é usada para codificar o texto.

O processo de treinamento normalmente envolve uma **perda contrastiva** , um tipo de função de perda que incentiva o modelo a aproximar exemplos positivos (pares semanticamente semelhantes) no espaço de incorporação, enquanto afasta exemplos negativos (pares diferentes).

Semelhante ao treinamento de LLMs, o treinamento de um modelo de incorporação do zero inclui duas etapas:

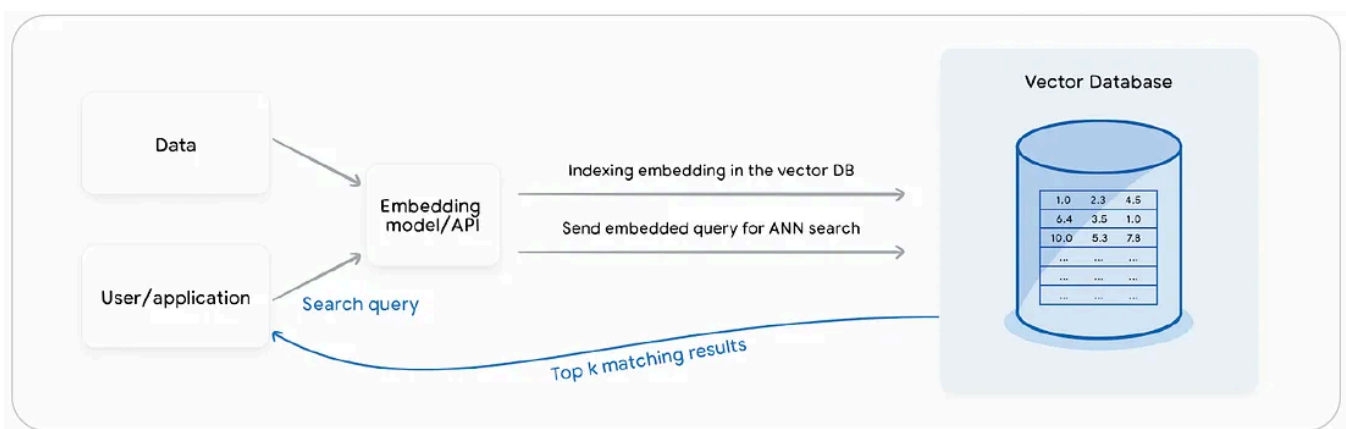
1. **Pré-treinamento:** nesta fase não supervisionada, o modelo aprende representações gerais de uma grande quantidade de dados não rotulados.
2. **Ajuste fino:** esta fase supervisionada envolve o treinamento do modelo em um conjunto de dados menor e rotulado, específico para a tarefa subsequente. O ajuste fino ajuda a adaptar as representações do modelo às nuances específicas da aplicação-alvo.

Lembre-se de que, para dados não estruturados, embeddings pré-treinados geralmente estão disponíveis e podem ser usados diretamente. No entanto, talvez seja necessário ajustá-los para aplicações específicas de domínio.

Bancos de dados vetoriais

Uma vez criados os embeddings, o próximo desafio é armazená-los e recuperá-los com eficiência. Historicamente, os bancos de dados tradicionais não dispunham dos meios para combinar significado semântico e consultas eficientes, de forma que os embeddings mais relevantes pudessem ser armazenados, consultados e recuperados de forma segura, escalável e flexível. Foi isso que deu origem aos bancos de dados vetoriais: **sistemas especializados projetados para gerenciar e consultar embeddings em escala**.

Cada banco de dados vetorial difere em sua implementação, mas o **fluxo geral** é mostrado na figura a seguir:



1. Um modelo de incorporação treinado apropriado é usado para incorporar os pontos de dados relevantes como vetores com dimensões fixas.
2. Os vetores são então aumentados com metadados apropriados e informações complementares (como tags) e indexados usando o algoritmo especificado para uma pesquisa eficiente.
3. Uma consulta recebida é incorporada ao mesmo modelo e usada para consultar e retornar quantidades específicas dos itens semanticamente mais semelhantes e seus respectivos conteúdos/metadados não incorporados. Alguns bancos de dados podem oferecer recursos de cache e pré-filtragem (com base em tags) e pós-filtragem (reclassificação usando outro modelo mais preciso) para aprimorar ainda mais a velocidade e o desempenho da consulta.

Alguns bons exemplos de bancos de dados de vetores gerenciados **comercialmente** são

- Pesquisa de vetores de IA do Google Vertex

- Pinha

E alguns exemplos de código aberto são:

- Weaviate
- Croma

Pesquisa vetorial

A busca vetorial é a alma dos bancos de dados vetoriais. É o **mecanismo que permite que esses bancos de dados liberem o poder da similaridade semântica e forneçam resultados altamente relevantes**.

Os algoritmos tradicionais de busca vetorial dependiam da correspondência de palavras-chave, o que os tornava lentos, difíceis de escalar e incapazes de retornar resultados que não correspondessem exatamente às palavras-chave da consulta. Em contraste, **as técnicas modernas de busca vetorial utilizam embeddings para recuperar informações com base na similaridade semântica, usando métricas como distância euclidiana, similaridade de cosseno ou produto escalar**. Em outras palavras, os métodos de busca vetorial identificam os itens no banco de dados que são mais semelhantes à consulta fornecida com base em suas representações vetoriais.

Pesquisa aproximada do vizinho mais próximo

Uma das técnicas mais populares para busca vetorial é a Busca por Vizinho Mais Próximo (RNA). RNA refere-se a um conjunto de algoritmos projetados para **identificar com eficiência os vetores mais próximos (mais semelhantes) em espaços de alta dimensão**.

Por que RNAs?

Em bancos de dados vetoriais (espaços de alta dimensão), o custo computacional das buscas exatas de vizinho mais próximo (NN) aumenta exponencialmente com o tamanho do conjunto de dados e o número de dimensões. Em vez disso, o que os algoritmos de RNA fazem é trocar precisão exata por velocidade e escalabilidade, entregando resultados "bons o suficiente" em uma fração do tempo.

Algoritmos ANN populares incluem:

1. Pequenos Mundos Navegáveis Hierárquicos (HNSW)

Pontos fortes :

- Extremamente rápido com alta precisão.
- Escalável para grandes conjuntos de dados.
- Flexível na adaptação a diferentes medidas de similaridade.

Casos de uso : tarefas ANN de uso geral, sistemas de recomendação e recuperação de vetores de alta dimensão.

Ferramentas : Integradas em bibliotecas como FAISS e nmslib .

2. Vizinho mais próximo aproximado escalável (ScaNN)**Pontos fortes :**

- Otimizado para conjuntos de dados em escala industrial.
- Combina particionamento, cálculo de distância aproximada e repontuação opcional para compensações de velocidade e precisão de alto nível.
- Lida com vetores densos e esparsos de forma eficaz.

Casos de uso : aplicativos em escala do Google, como pesquisa e recuperação de incorporações.

Ferramentas: ScaNN .

3. Annoy (vizinhos mais próximos aproximados)**Pontos fortes :**

- Leve e simples de usar.
- Otimizado para armazenamento mapeado em memória, o que o torna ótimo para conjuntos de dados estáticos.

Casos de uso : sistemas de recomendação, especialmente para conjuntos de dados menores ou casos de uso em que a eficiência da memória é fundamental.

Ferramentas : Irritar .

4. FAISS (pesquisa de similaridade de IA do Facebook)**Pontos fortes :**

- Biblioteca abrangente com suporte a vários algoritmos ANN, incluindo HNSW, IVFPQ (Índice de arquivo invertido com quantização de produto) e LSH.
- Altamente otimizado para CPU e GPU, tornando-o ideal para pesquisas em larga escala.

Casos de uso : grandes conjuntos de dados com bilhões de vetores, recomendações em tempo real.

Ferramentas : FAISS .

5. Hashing Sensível à Localidade (LSH)

Pontos fortes :

- Excelente para dados esparsos de alta dimensão.
- Simples de implementar e bem compreendido.

Casos de uso : pesquisa de texto, dados genômicos ou qualquer caso de uso que exija pesquisa de similaridade baseada em raio.

Ferramentas : disponíveis em bibliotecas como scikit-learn e datasketch .

Caso de uso: Construindo um LLM intensivo em conhecimento com bancos de dados RAG e vetoriais

Imagine que você está criando um chatbot de suporte ao cliente que precisa fornecer respostas precisas e atualizadas às dúvidas dos clientes. Para isso, você pode utilizar uma combinação de LLMs, Geração Aumentada de Recuperação (RAG) e bancos de dados vetoriais.

Aqui está uma análise do processo:

1. Preparação de dados:

- **Coleta de documentos:** reúna documentos relevantes, como perguntas frequentes, manuais de produtos e registros de suporte ao cliente.
- **Vetorização:** Use um modelo de linguagem (por exemplo, BERT, RoBERTa) para converter cada documento em uma representação vetorial densa. Esse vetor captura o significado semântico do texto.

2. Banco de dados de vetores:

- **Indexação:** Armazene esses vetores em um banco de dados de vetores (por exemplo, Pinecone, Weaviate, Chroma). Este banco de dados é otimizado para uma busca de similaridade eficiente.

3. Sistema RAG:

- **Processamento de consulta:** quando um cliente faz uma pergunta, converta-a em uma representação vetorial usando o mesmo modelo de linguagem.

- **Recuperação:** consulte o banco de dados de vetores para encontrar os documentos mais semelhantes ao vetor de consulta.
- **Aumento:** combine os documentos recuperados com a consulta original e insira-os no LLM.

4. Geração de Resposta LLM:

- O LLM processa o prompt aumentado e gera uma resposta abrangente e informativa.
- A resposta pode ser uma resposta direta à consulta, um resumo de informações relevantes ou uma combinação de ambos.

Conclusão

Embeddings revolucionaram a maneira como trabalhamos com dados diversos em aprendizado de máquina. Sua capacidade de capturar relações semânticas, facilitar comparações eficientes e servir como entradas para diversos modelos abriu um mundo de possibilidades. Ao escolher a técnica de embedding, o banco de dados vetorial e o algoritmo de busca corretos, os desenvolvedores podem explorar todo o potencial dos embeddings para criar aplicativos poderosos.

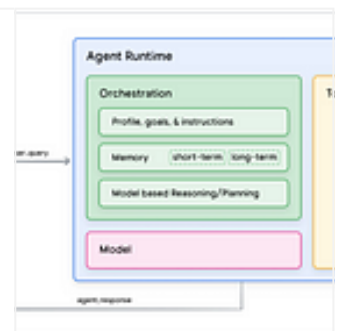
Casos de uso importantes, como busca semântica, sistemas de recomendação, detecção de anomalias e LLMs com RAG, demonstram a versatilidade dos embeddings para lidar com desafios complexos. À medida que a tecnologia continua a evoluir, os embeddings continuarão sendo um componente fundamental para o avanço de soluções baseadas em IA.

Se você tiver interesse em conteúdo relacionado, recomendo a leitura do meu próximo artigo, [Introdução aos Agentes de IA](#) . Ele explora o mundo dos agentes de IA, abordando sua arquitetura, componentes principais e implementação para aplicações do mundo real.

Introdução aos Agentes de IA

Arquitetura, Ferramentas e Implementação

medium.com



Referências

[1] Nawalgaria, A., & Ren, X. (2024). *Embeddings e armazenamentos de vetores*.

E é isso! Obrigado pela leitura! Espero que tenha gostado deste artigo. Se quiser, adicione-me no LinkedIn.

- Incorporação
- Banco de Dados Vetorial
- Pesquisa de vetores
- Llm
- IA



Seguir

Escrito por Aleix López Pascual

115 seguidores · 6 seguintes

Cientista de Dados Sênior na Glovo | Especialista em Competições na Kaggle | Escritor na Medium | Mestre em Física de Altas Energias, Astrofísica e Cosmologia

Respostas (1)



Thiago Pablicio

O que você pensa?



Tejas Kumar
4 de dezembro de 2024



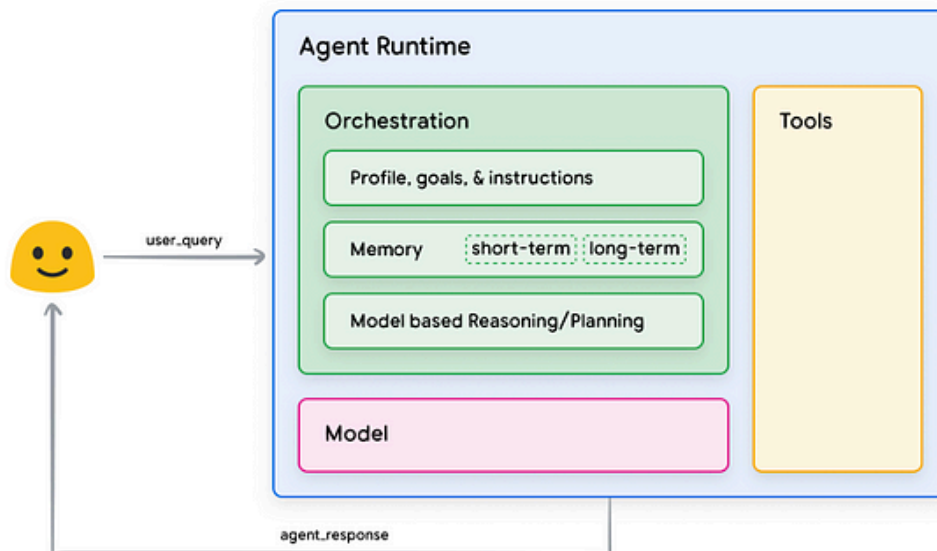
Ótimo texto, Aleix! Adorei os detalhes que você detalhou! Uma observação sobre o BERT: na DataStax (onde trabalho, também criamos um banco de dados vetorial chamado Astra DB), criamos um algoritmo de busca vetorial chamado "ColBERT" que combina o sinal de... [mais](#)



4

[Responder](#)

Mais de Aleix López Pascual



Aleix López Pascual

Introdução aos Agentes de IA

Arquitetura, Ferramentas e Implementação

12 de janeiro

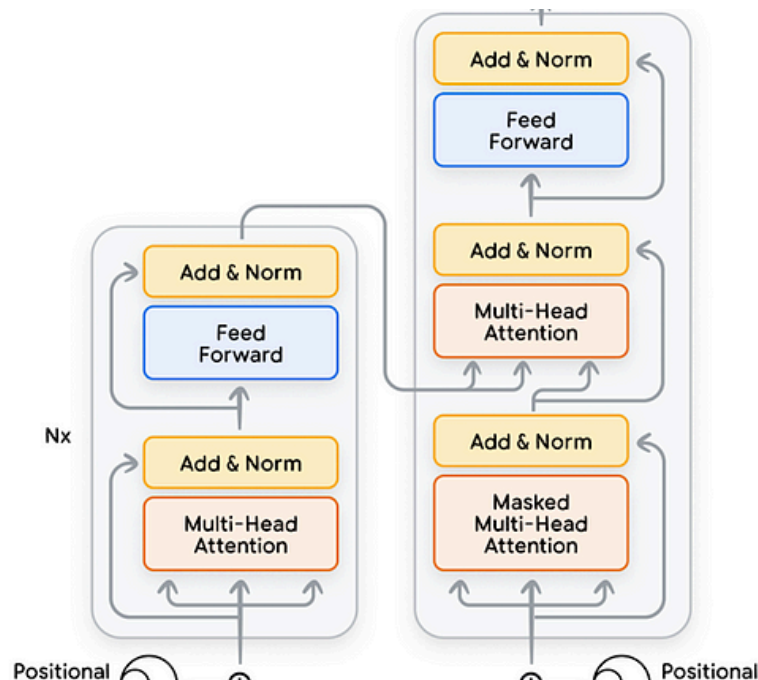


60



3





 Aleix López Pascual

Introdução aos Modelos Fundamentais de Grandes Linguagens

A arquitetura subjacente, metodologias de treinamento, técnicas de ajuste fino e métodos de otimização de inferência para o mundo real...

17 de novembro de 2024  56

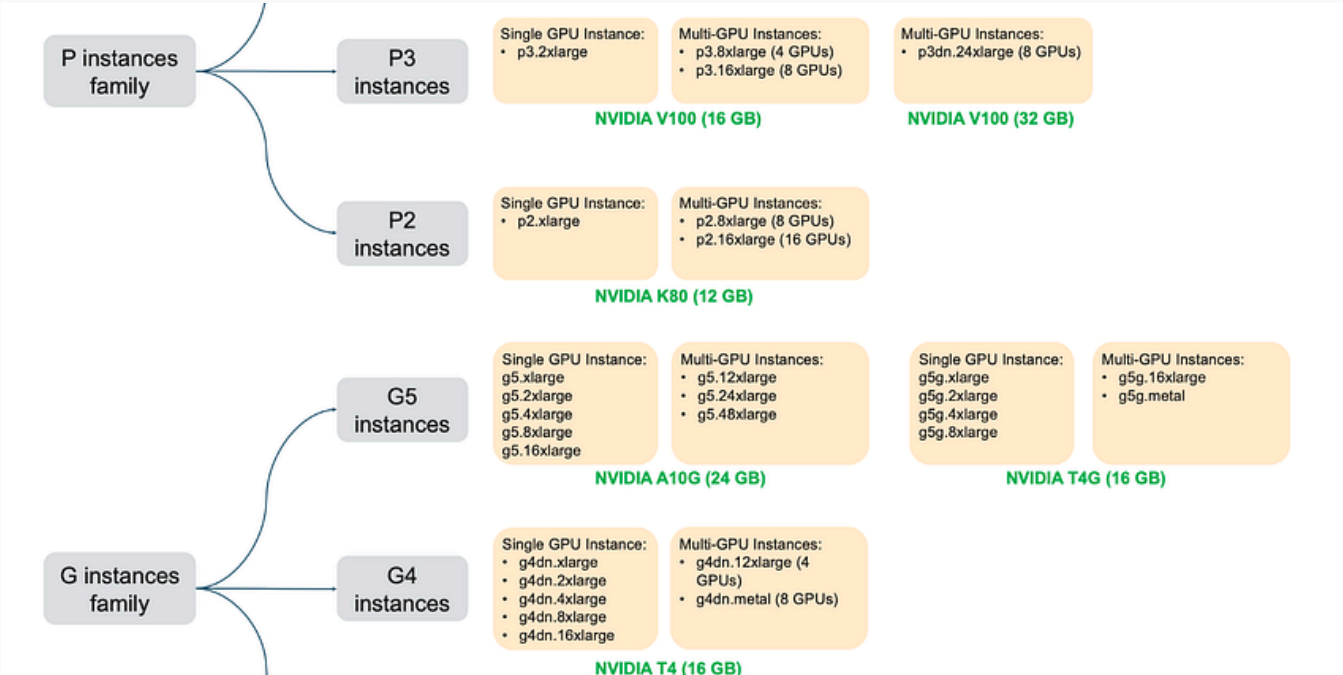
 

0.1	20	0.95	More deterministic and factual: The LLM will favour highly probable tokens, resulting in predictable and coherent output. However, some slight variation is still possible due to the low temperature and relatively high top-K and top-P values.
0.9	40	0.99	Highly creative and diverse: The high temperature encourages exploration of less probable tokens, and the high top-K and top-P values provide a wide range of options. Expect unexpected and imaginative outputs.
0	10	0.8	Completely deterministic (greedy decoding): The temperature of 0 forces the LLM to always select the single most likely token. Top-K and Top-P become irrelevant in this scenario.
0.5	1	0.5	Deterministic despite temperature (greedy decoding): Setting top-K to 1 forces the LLM to only consider the most probable token,

 Aleix López Pascual

Introdução à Engenharia Prompt

Dominando a arte da engenharia rápida: técnicas, melhores práticas e aplicações no mundo real



Aleix López Pascual

Instâncias de GPU em nuvem para resolver erro de falta de memória 2022

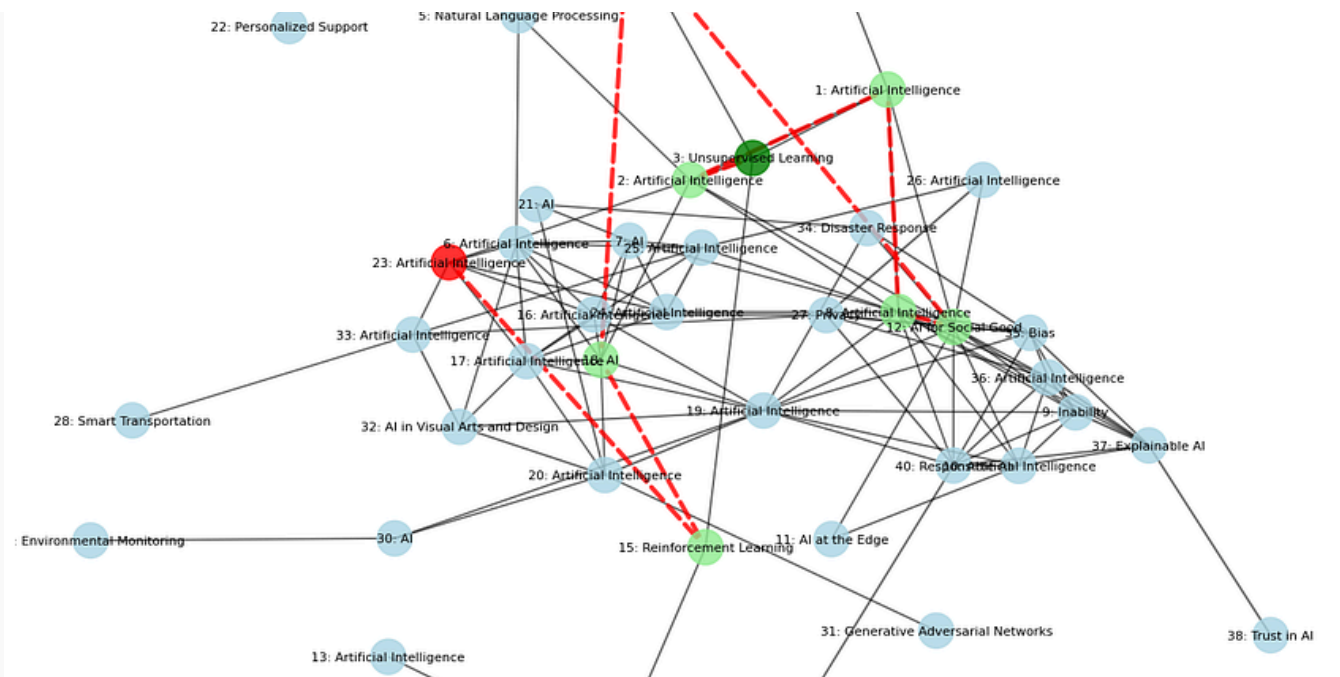
Descubra as GPUs com maior VRAM

14 de outubro de 2022 63 1

+

Ver tudo de Aleix López Pascual

Recomendado pelo Medium



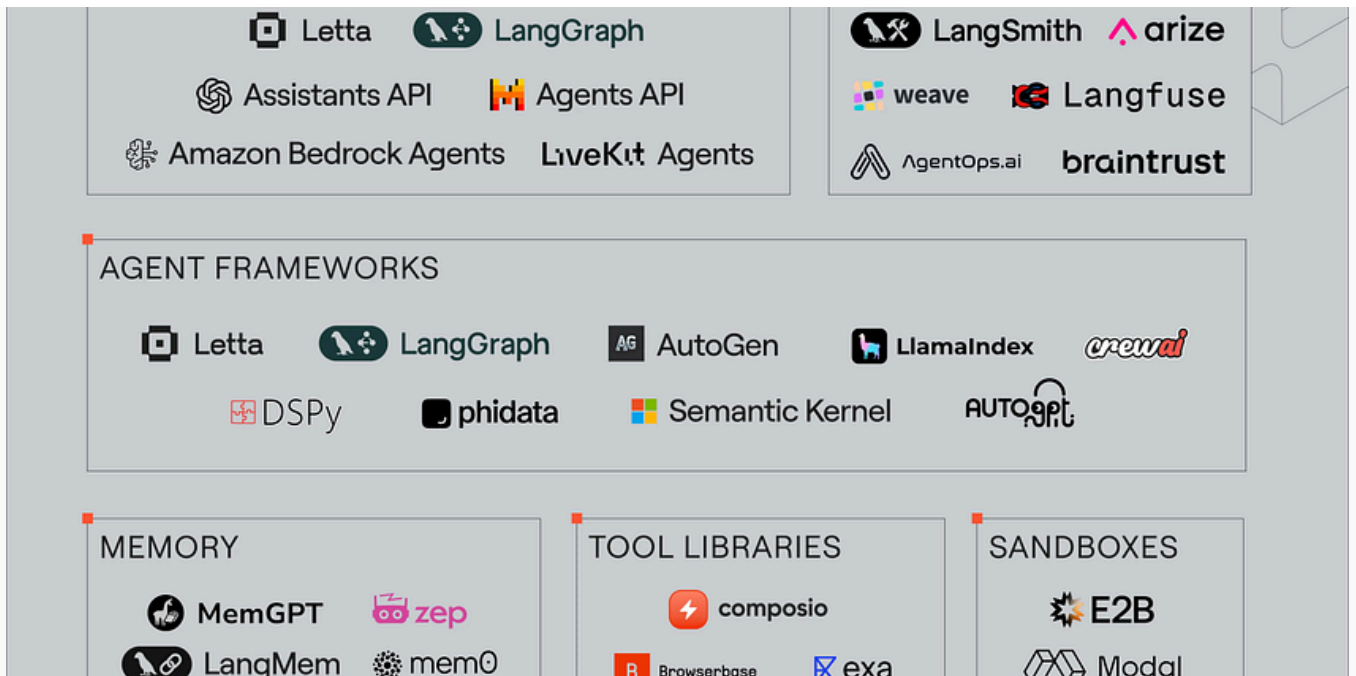
 Em Codificação de nível superior por Fareed Khan

Testando 18 técnicas RAG para encontrar a melhor

crag, HyDE, fusão e muito mais!

🌟 11 de março 🖐️ 2,2 mil 💬 31



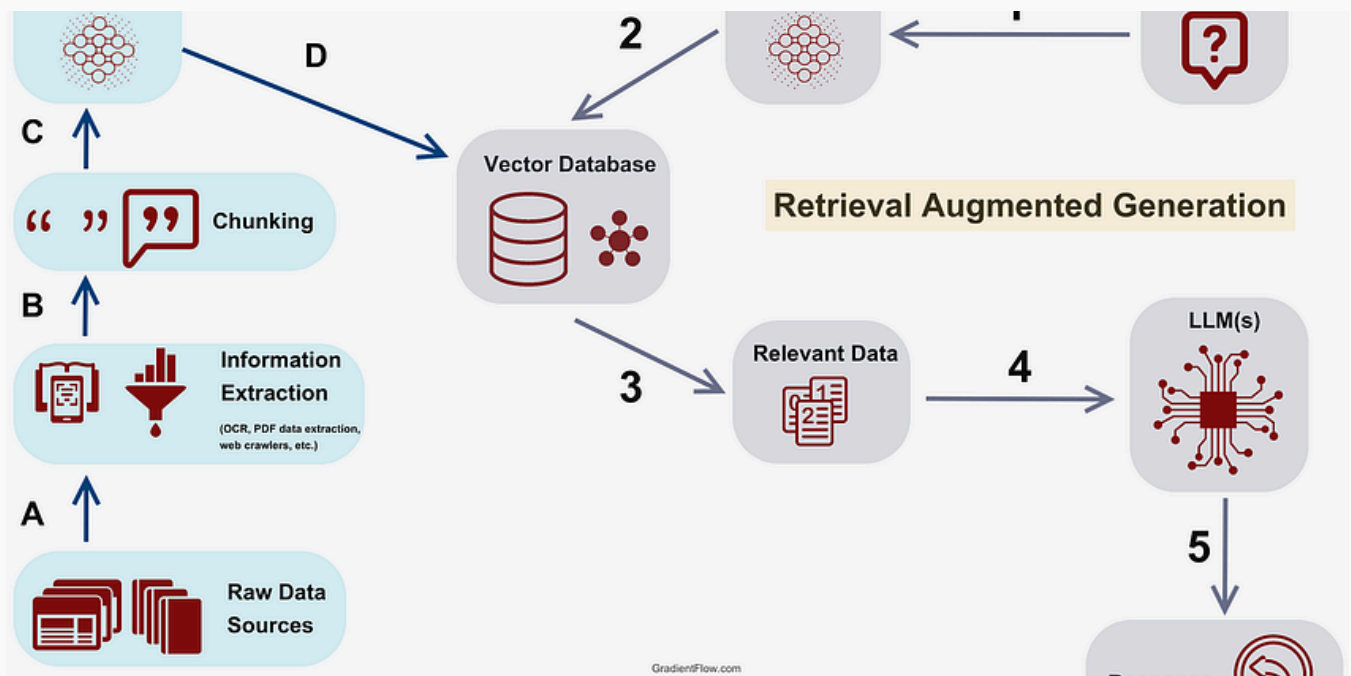


Vipra Singh

Agentes de IA: Introdução (Parte 1)

Descubra agentes de IA, seu design e aplicações no mundo real.

2 de fevereiro 2,7 mil 56



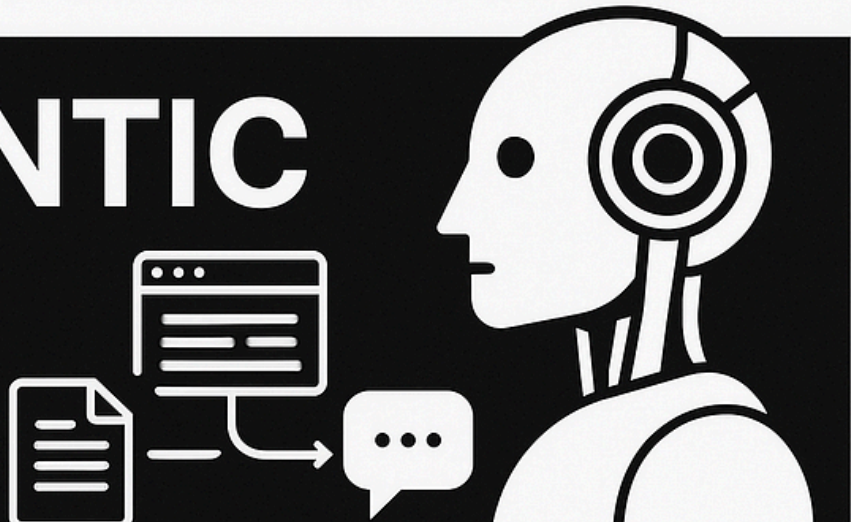
Em INEXT por Javier Ramos

Você não precisa do RAG! Crie um agente de IA para perguntas e respostas em 30 minutos 🚀

O RAG está morto? Explorando alternativas mais simples para agentes de IA, criando ferramentas que consultam os dados de origem diretamente.

Hands on

AGENTIC RAG



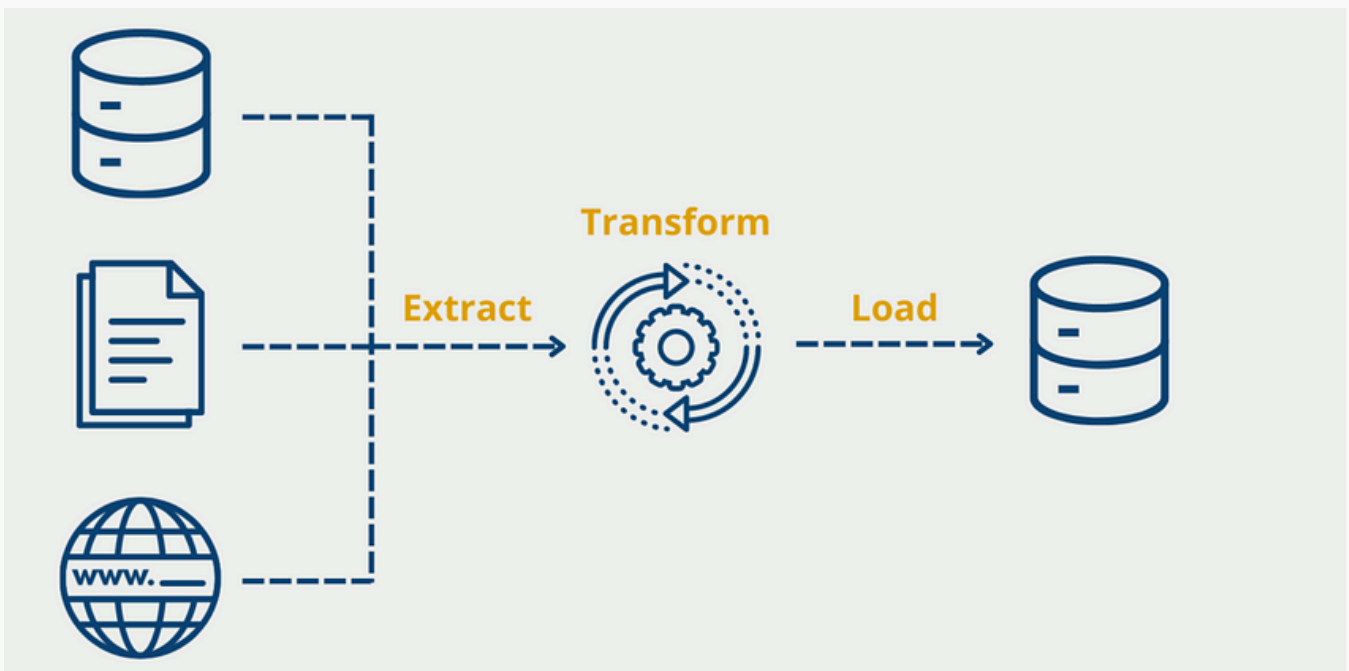
 AB Vijay Kumar 🏠

Mãos à obra — Agentic RAG (1/3)

Neste blog, comparo o RAG tradicional com o Agentic RAG e faço um tutorial rápido sobre como construir um aplicativo Agentic RAG simples com...

11 de junho 🖱️ 15 💬 2

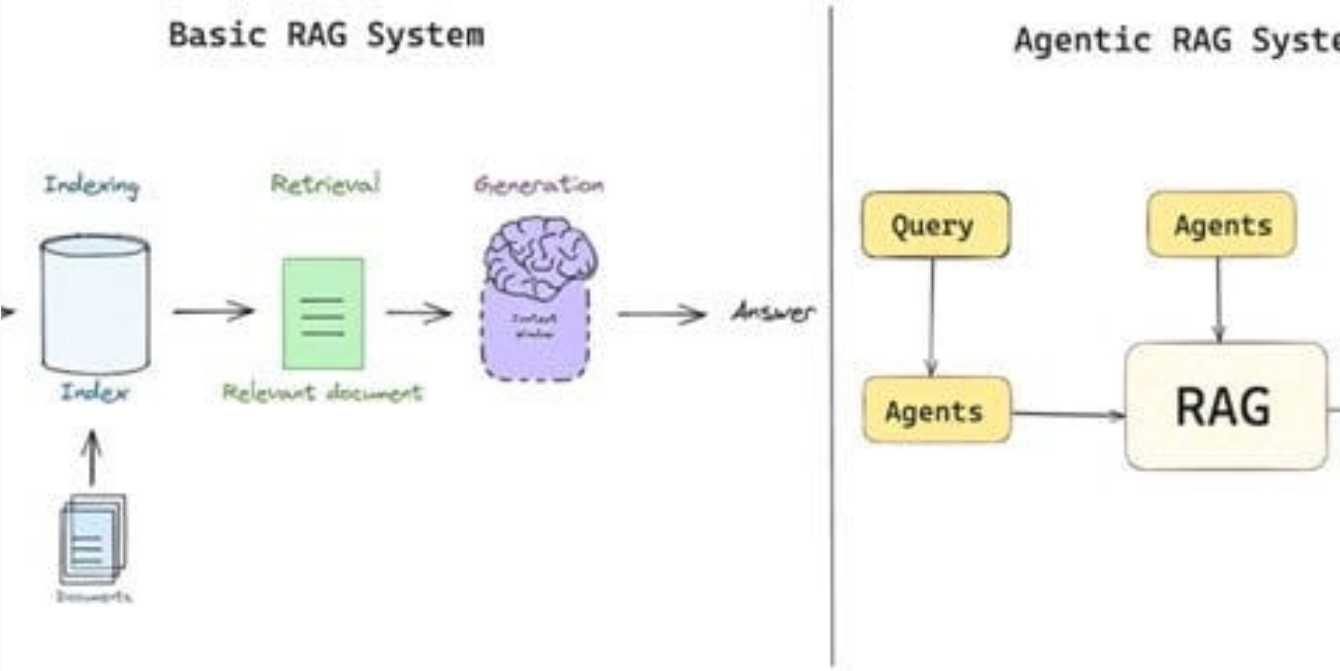
🔖+ ...



O Fim do ETL: A Mudança Radical no Processamento de Dados que Está por Vir

O ETL está morrendo. Não lentamente, nem silenciosamente, mas em um surto espetacular de irrelevância que a maioria das pessoas ainda não percebeu.

🌟 13 de junho 🖱️ 1,5 mil 💬 60 🔖 ...



 Em Gênio do Desenvolvimento por Nishad Ahamed

RAG vs Agentic RAG: Uma comparação abrangente

Introdução

🌟 6 de março 🖱️ 4 🔖 ...

Veja mais recomendações